

面向对象程序设计

Web前端基础

赵毅力

大数据与智能工程学院

西南林业大学

Web开发



HTML标签

- HTML页面的基本结构
- HTML页面中的元素以标签的形式表示

```
<div class="swfu">  
  <p>西南林业大学</p>  
</div>
```

HTML元素的属性

- 一个元素的 **class** 属性可为元素提供一个标识名称，以便进一步为元素指定样式或进行其他操作时使用。

```
<p class="logo">This is Southwest Forestry University logo</p>
```

空元素

- **img** 元素包含两个属性，但是并没有 `</img>` 结束标签，元素里也没有内容。这是因为图像元素不需要通过内容来产生效果，它的作用是向其所在的位置嵌入一个图像。

```

```

常用的元素标签

```
<h1></h1>
```

```
<p></p>
```

```
<div></div>
```

```
<ul></ul>
```

```
<ol></ol>
```

```
<a></a>
```

CSS基础

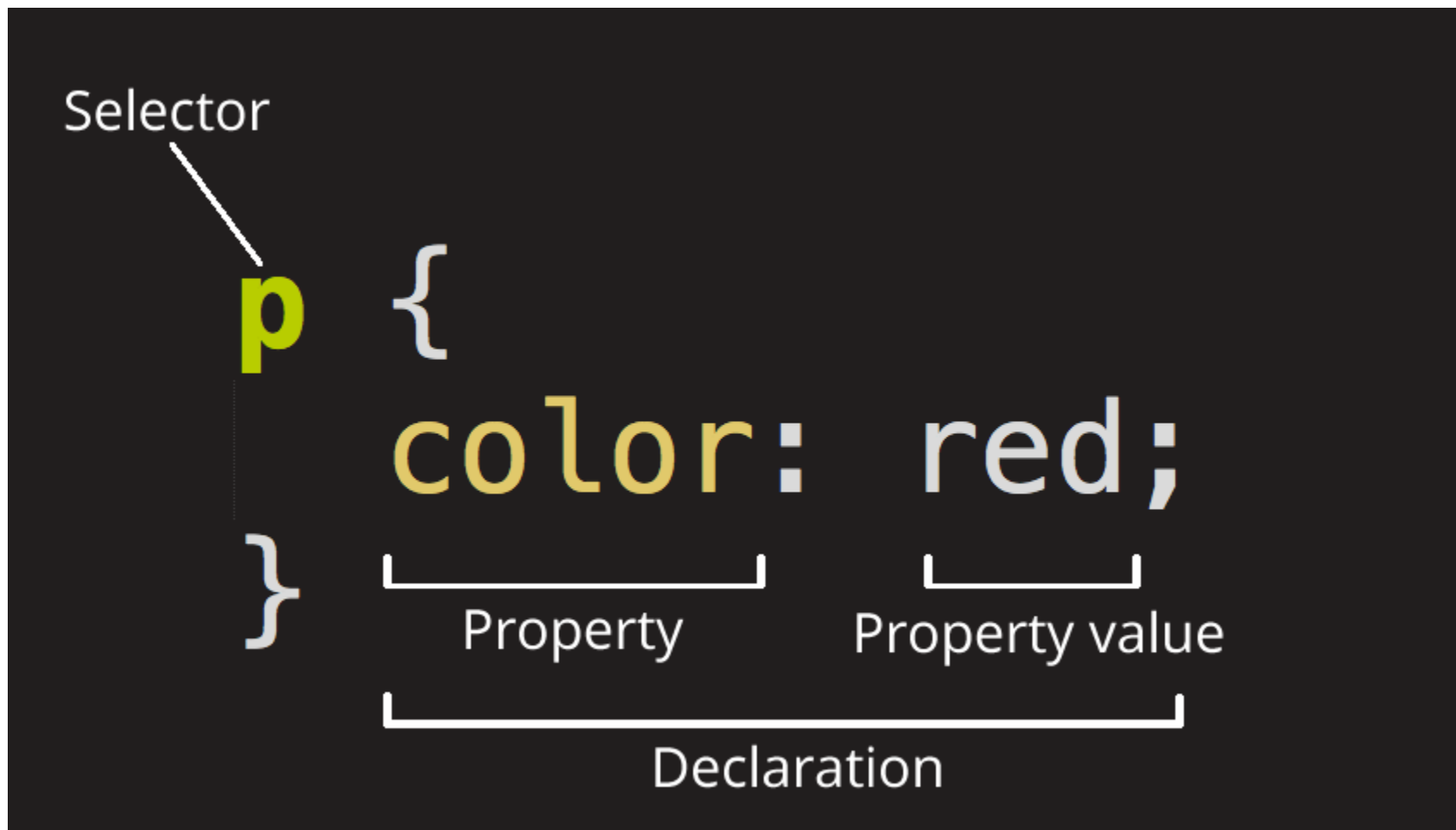
- CSS（Cascading Style Sheets，层叠样式表）：为 Web 内容添加样式的代码。
- CSS 是一门样式表语言，可以用它来选择性地为 HTML 元素添加样式。

CSS颜色

颜色	示例
十六进制	<pre>#RRGGBB p { background-color: #FF0000; /* 红色 */ }</pre>
RGB	<pre>RGB(红色分量,绿色分量,蓝色分量) p { background-color: RGB(255,0,0); }</pre>
浏览器预定义	<pre>颜色名称 p { background-color: Red; }</pre>

“CSS 规则集”详解

- 在CSS 中，选择器用于选择要应用样式的 HTML 元素。



CSS

- 如果要同时修改多个属性，只需要将它们用**分号**隔开：

```
p {  
  color: red;  
  width: 500px;  
  border: 1px solid black;  
}
```

选择多个元素

- 也可以选择多种类型的元素并为它们添加一组相同的样式。将不同的选择器用**逗号**分开：

```
p,  
li,  
h1 {  
    color: red;  
}
```

CSS选择器

选择器名称	选择内容	示例
元素选择器	通过元素名称选择 HTML 元素	<code>p</code> 选择所有<p>元素
ID选择器	通过元素的唯一标识符（ID）选择 HTML 元素	<code>#logo</code> 选择 <code><p id="logo"></code> 或 <code></code>
类选择器	通过类别名称选择具有特定类别的 HTML 元素。	<code>.desc</code> 选择 <code><p class="desc"></code> 和 <code></code>
属性选择器	通过元素的属性选择 HTML 元素。属性选择器可以根据属性名和属性值进行选择。	<code>img[src]</code> 选择 <code></code> 但不是 <code></code>
伪类选择器	特定状态下的特定元素（比如鼠标指针悬停于链接之上）	<code>a:hover</code> 选择仅在鼠标指针悬停在 链接上时的 <code><a></code> 元素

CSS选择器

- **tagname**: 通过标签查找元素, 比如: a
- **#id**: 通过ID查找元素, 比如: #logo
- **.class**: 通过class名称查找元素, 比如: .roleIntroduction
- **[attribute]**: 利用属性查找元素, 比如: [href]

CSS组合选择器

- **el#id**: 元素+ID, 比如: `div#logo`
- **el.class**: 元素+class, 比如: `div.roleIntroduction`
- **el[attr]**: 元素+属性, 比如: `a[href]`
- **ancestor child**: 查找某个元素的子元素, 比如: 可以用`.body p`查找在"body"元素下的所有p元素
- **parent > child**: 查找某个父元素的直接子元素, 比如: 可以用`div.content > p`查找class属性是"content"的"div"元素的直接 p 元素

如何使用JavaScript选择HTML页面中的元素？

- 当浏览器对HTML页面渲染完成以后，会自动生成一个 document 对象。
- **document.querySelector(元素名称);**
- **document.getElementById(元素ID);**
- **document.getElementsByClassName(元素的类名称)**

Web开发平台

- 在进行Web开发时，可以把浏览器看作一个开发平台。
- 浏览器提供很多内置函数，作为开发的接口。
- 开发者可以使用JavaScript语言调用这些函数。
- `document.querySelector` 和 `alert` 是浏览器内置的函数，可以直接使用。

JavaScript

JavaScript语言也是一门面向对象程序设计语言。

但是，JavaScript的面向对象是基于原型（Prototype）的。

TypeScript语言使得JavaScript具有类似Java的面向对象。

JavaScript变量和常量

- 使用 **let** 定义一个变量
 - `let number = 2.7;`
- 使用 **const** 定义一个常量
 - `const name = "John";`

JavaScript数据类型

变量	解释	示例
String	字符串：字符串的值必须用引号（单双均可，必须成对）括起来。	let name = '李小明';
Number	数值类型	let number = 2.72;
Boolean	布尔值（true/ false）	let isAlive = true;
Array	数组	let fruits = ['apple', 'grape']; 元素引用方法：fruits[0]
Object	对象：JavaScript 里一切皆对象（函数也是对象）	let h1element = document.querySelector('h1');

JavaScript字符串

- 用双引号表示字符串
 - `const name = "John";`
- 字符串模板
 - 模板字符串是用反引号 (```) 分隔的字面量。

```
const name = "John";  
const msg = `Hello ${name}!`;  
console.log(msg)
```

JavaScript模板字符串

- 模板字符串是用反引号 (``) 分隔的字面量。
- 有三个[0,255]之间的随机数： 229,153,29
- 通过这个三个随机数合成一个字符串： 'RGB(229,153,29)'

```
let red = 229  
let green = 153  
let blue = 29
```

占位符：由美元符号和大括号分隔

```
const rndColor = `RGB(${red}, ${green}, ${blue})`  
console.log((rndColor))
```

字符串常用函数

`split()`: 通过搜索模式将字符串分割成一个有序的子串列表，将这些子串放入一个数组，并返回该数组。

`replace()`: 将源字符串替换为目标字符串，并返回新形成的字符串。

`join()`: 将所有数组元素连接成一个单一的字符串，并返回这个新字符串。

JavaScript数组

- 使用 [] 定义数组

```
const seasons = [ "Spring", "Summer", "Autumn", "Winter" ];
```

JavaScript循环

- for循环

```
let seasons = [ "Spring", "Summer", "Autumn", "Winter" ];
```

```
// 方式一
```

```
for (let i = 0; i < seasons.length; i++) {  
    console.log(seasons[i]);  
}
```

```
// 方式二
```

```
for (let s of seasons) {  
    console.log(s);  
}
```


JavaScript函数

- 使用关键字 **function** 定义函数

```
// 返回一个[0,number-1]之间的随机整数  
function random(number) {  
    return Math.floor(Math.random() * number);  
}
```

- 备注： Math.random() 函数只生成一个 [0.0,1.0)之间的随机小数。

函数的默认参数

```
function hello(name = "李小明") {  
    console.log(`你好, ${name}!`);  
}
```

```
hello("张明");  
hello();
```

箭头函数

- 将数组中的每个元素变为自身的平方

```
function doubleItem(item) {  
  return item * item;  
}  
  
const ns = [1, 3, 5, 7, 9];  
const nsd = Array(ns.length).fill(0);  
for (let i = 0; i < nsd.length; i++) {  
  nsd[i] = doubleItem(ns[i]);  
}  
console.log(nsd);
```



```
const ns = [1, 3, 5, 7, 9];  
const nsd = ns.map(e => e * e);  
console.log(nsd);
```

备注：map() 方法依次获取数组中的每一项，并将其传递给给定函数。然后，它将该函数返回的值添加到一个新数组中。

事件处理

- Web的事件由浏览器负责定义，开发者使用JavaScript语言对事件进行处理。

为元素添加事件处理

在一个HTML页面中，能够触发事件的元素都有 **addEventListener()** 方法成员。

可以使用 **addEventListener()** 为某个元素添加事件处理。

使用 addEventListener() 方法

第1个参数：
需要监听的事件名称

第2个参数：对事件
进行处理的函数名称
(也称为回调函数)

```
btn.addEventListener("click", changeBackgroundColor);
```

使用箭头函数对事件进行处理

```
const btn = document.querySelector("button");

function random(number) {
  return Math.floor(Math.random() * (number + 1));
}

btn.addEventListener("click", () => {
  const rndCol = `rgb(${random(255)}, ${random(255)}, ${random(255)})`;
  document.body.style.backgroundColor = rndCol;
});
```

使用命名函数对事件进行处理

```
const btn = document.querySelector("button");

function random(number) {
  return Math.floor(Math.random() * (number + 1));
}

function changeBackgroundColor() {
  const rndCol = `rgb(${random(255)}, ${random(255)}, ${random(255)})`;
  document.body.style.backgroundColor = rndCol;
}

btn.addEventListener("click", changeBackgroundColor);
```



```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet" href="src/style.css">
  </head>
  <body>
    <div>
      <h1 id="header"></h1>
      <button>按钮</button>
    </div>
    <script src="src/script.js"></script>
  </body>
</html>
```

```
const btn = document.querySelector("button")

function random(number) {
  return Math.floor(Math.random() * (number + 1))
}

btn.addEventListener("click", () => {
  const rndColor = `rgb(${random(255)},
${random(255)}, ${random(255)})`
  document.body.style.backgroundColor = rndColor
});
```